

Implementación de Auditoría de Operaciones

Actividad N°: 26 **Fecha:** 06/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Completar el módulo de auditoría (RF-08), asegurando que toda operación relevante quede registrada de forma automática y consultable, sin acoplar esa responsabilidad a la lógica de negocio de cada controlador.

2. Middleware `auditLogger` (registro automático desacoplado)

`backend/src/middleware/auditLogger.js` implementa un patrón de **interceptor de respuesta**: envuelve `res.json` para registrar el log de auditoría **después** de que la operación principal ya respondió exitosamente, sin bloquear al usuario:

```
const auditLogger = (tipo) => (req, res, next) => {
  const originalJson = res.json;
  res.json = function(data) {
    if (res.statusCode < 400 && req.user) {
      setImmediate(async () => {
        // construir logData según "tipo" y crear AuditLog
        await AuditLog.create(logData);
      });
    }
    return originalJson.call(this, data);
  };
  next();
};
```

Decisiones de diseño clave:

- Se usa `setImmediate` para que la escritura del log ocurra en el siguiente ciclo del event loop, **después** de enviar la respuesta HTTP al cliente — el usuario no espera al log para recibir su confirmación de canje/impresión.
- Solo se registra si `res.statusCode < 400`, es decir, únicamente operaciones exitosas.
- El middleware se reutiliza en distintas rutas simplemente parametrizando el `tipo` ('canje', 'canje_masivo', 'impresion', 'reimpresion', 'creacion_usuario'), sin duplicar código de registro en cada controlador.

3. Endpoints de consulta de auditoría (`auditController.js`)

Endpoint	Función
GET /api/audit	Lista paginada de logs, con filtros por <code>tipo</code> , <code>usuario</code> , <code>ticketId</code> y rango de fechas (<code>fechaInicio/fechaFin</code>)

Endpoint	Función
GET <code>/api/audit/summary</code>	Resumen agregado: logs por tipo, logs por usuario (con <code>\$lookup</code> a <code>users</code> para traer nombre y rol), logs por día

Ambos endpoints están restringidos a `authorize('jefe')`.

4. Doble vía de registro de auditoría

Se identifican dos mecanismos complementarios que coexisten en el sistema:

1. **Vía middleware genérico** (`auditLogger`): usado en rutas de tickets y creación de usuarios, captura automáticamente el contexto del request (`req.body`, `req.user`).
2. **Vía registro manual dentro del controlador** (`AuditLog.create(...)` directo): usado en `authController.js` para `login`, `logout` y `cambio_password`, y en `canjeTicket/bulkCanjeTickets` para incluir detalles de negocio más específicos (ej. `parentesco`, `quienOtro`, `bulkOperation`).

Ambas vías escriben sobre el mismo modelo `AuditLog`, por lo que la consulta y el resumen (`getAuditLogs/getAuditSummary`) funcionan de manera uniforme sin importar cuál mecanismo generó cada registro.

5. Frontend — `AuditPage.jsx`

- Tabla de logs con columnas: tipo de operación, usuario (nombre y rol vía `populate`), ticket/transacción afectada, punto de trabajo y fecha.
- Filtros por tipo de operación y por usuario.
- Acceso exclusivo para el rol Jefe (protegido en frontend y backend).

6. Conclusiones del día

El módulo de auditoría queda completo: cada operación sensible del sistema (autenticación, gestión de usuarios, canje, impresión) genera un registro trazable, ya sea vía el middleware genérico o vía registro manual con detalle de negocio, y ambos son consultables desde los mismos endpoints de reporte.

Observaciones: Sin observaciones.

Desarrollo de Reportes y Estadísticas

Actividad N°: 27 **Fecha:** 07/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Implementar los reportes y estadísticas agregadas que alimentan el dashboard administrativo (RF-11), usando el framework de agregación de MongoDB para calcular métricas sin traer todos los documentos al backend.

2. Estadísticas de tickets (`ticketController.getTicketStats`)

Métrica	Cálculo
<code>totalTickets</code>	<code>countDocuments()</code> sobre toda la colección activa
<code>ticketsCanjeados</code>	<code>countDocuments({ canjeado: true, ...filtros })</code>
<code>ticketsRestantes</code>	<code>totalTickets - ticketsCanjeados</code>
<code>porcentajeCanjeados</code>	$(\text{ticketsCanjeados} / \text{totalTickets}) * 100$, redondeado a 2 decimales
<code>evolucionDiaria</code>	Agregación agrupando por día (<code>\$dateToString</code>) los canjes realizados, ordenados cronológicamente
<code>ticketsPorPunto</code>	Agregación agrupando canjes por <code>puntoTrabajo</code> , ordenados de mayor a menor

Filtros soportados: `puntoTrabajo`, `fechaInicio`, `fechaFin` (sobre `fechaCanje`). Todas las consultas de esta función se ejecutan en paralelo con `Promise.all` para minimizar el tiempo total de respuesta.

3. Resumen de auditoría (`auditController.getAuditSummary`)

Métrica	Cálculo
<code>logsPorTipo</code>	Agrupación por <code>tipo</code> de operación (<code>canje</code> , <code>impresion</code> , <code>login</code> , etc.)
<code>logsPorUsuario</code>	Agrupación por <code>usuario</code> , con <code>\$lookup</code> a la colección <code>users</code> para incluir nombre y rol en el resultado, ordenado por cantidad descendente
<code>logsPorDia</code>	Agrupación por día (<code>\$dateToString</code>) de todos los eventos de auditoría

Al igual que en `getTicketStats`, las tres agregaciones se ejecutan en paralelo.

4. Ejemplo de pipeline de agregación (evolución diaria)

```
Ticket.aggregate([
  { $match: { canjeado: true, ...matchQuery } },
  { $group: {
```

```
    _id: { $dateToString: { format: "%Y-%m-%d", date: "$fechaCanje" } },  
    count: { $sum: 1 }  
  }},  
  { $sort: { _id: 1 } }  
]);
```

Este patrón se repite (con variaciones de campo de agrupación) tanto para tickets como para logs de auditoría, evitando traer todos los documentos individuales al servidor de aplicación solo para contarlos o agruparlos en memoria.

5. Diferencia entre "reportes de negocio" y "reportes de auditoría"

- **Reportes de negocio** (`getTicketStats`): responden la pregunta "*¿cómo va el evento?*" — cuántos boletos canjeados, cuántos faltan, evolución en el tiempo, distribución por punto de trabajo.
- **Reportes de auditoría** (`getAuditSummary`): responden la pregunta "*¿quién hizo qué y cuándo?*" — enfocados en control interno y trazabilidad, no en el avance operativo del evento.

Ambos se exponen como endpoints separados y restringidos a Jefe, pero se combinan visualmente en el Dashboard y en la página de Auditoría respectivamente.

6. Conclusiones del día

Quedan implementados los reportes agregados de tickets y de auditoría usando el framework de agregación de MongoDB, calculando las métricas del lado de la base de datos en vez de procesarlas en memoria en Node.js, lo cual es más eficiente para volúmenes grandes de datos.

Observaciones: Sin observaciones.

Implementación de Dashboard Administrativo

Actividad N°: 28 **Fecha:** 08/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

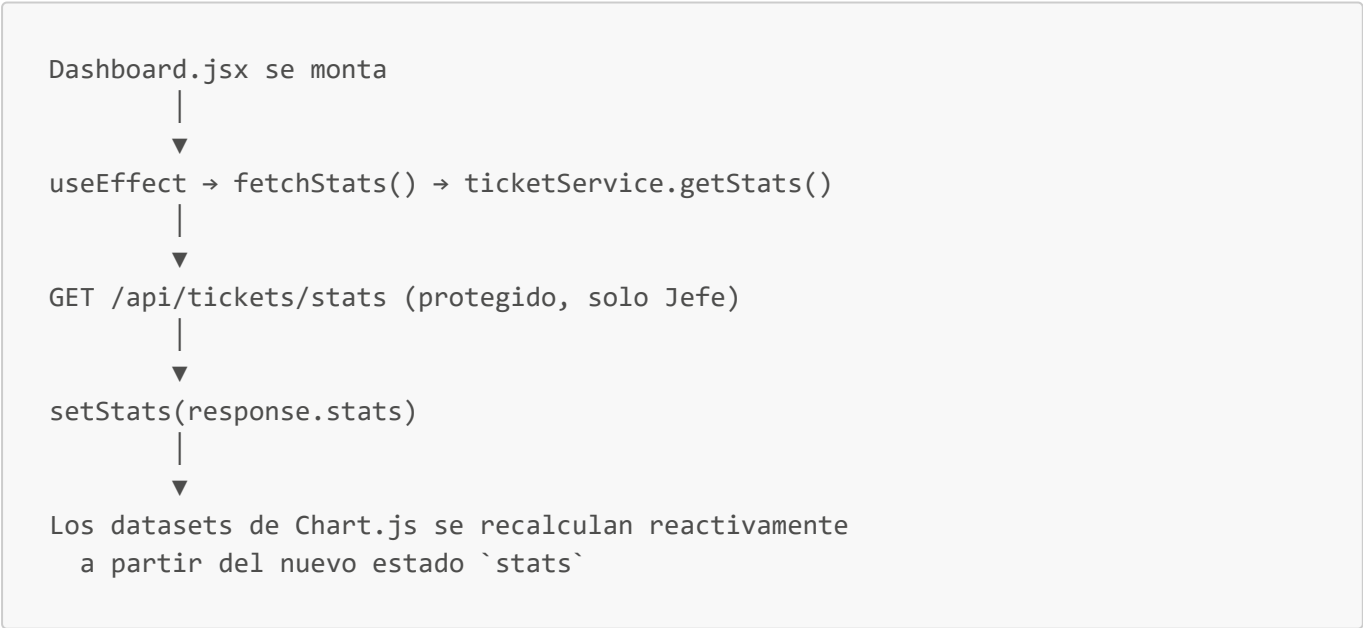
1. Objetivo del día

Construir la interfaz del Dashboard administrativo (**Dashboard.jsx**), visualizando las estadísticas desarrolladas el día anterior mediante gráficos accesibles para el rol Jefe.

2. Componentes visuales implementados

Componente	Librería	Datos que consume
Gráfico de dona (Canjeados vs. Pendientes)	Chart.js (Doughnut)	<code>stats.ticketsCanjeados</code> , <code>stats.ticketsRestantes</code>
Gráfico de línea (Evolución diaria de canjes)	Chart.js (Line)	<code>stats.evolucionDiaria</code> (array de <code>{ _id: fecha, count }</code>)
Tarjetas resumen	React Bootstrap (Card)	Totales, porcentaje de avance
Distribución por punto de trabajo	Tabla/gráfico adicional	<code>stats.ticketsPorPunto</code>

3. Flujo de carga de datos



4. Configuración de los gráficos

- Dona:** colores semánticos — verde (`#28a745`) para canjeados, amarillo (`#ffc107`) para pendientes, coherente con la convención de colores ya usada en la tabla de tickets (fila verde = canjeado).

- **Línea:** eje X con las fechas de `evolucionDiaria`, eje Y iniciando siempre en cero (`beginAtZero: true`) para no distorsionar visualmente la magnitud del crecimiento diario.
- Ambos gráficos son `responsive: true`, adaptándose a distintos tamaños de pantalla (el dashboard puede consultarse desde una tablet en el punto de control del evento).

5. Filtros disponibles en el Dashboard

Consistente con `getTicketStats`, la interfaz permite filtrar las estadísticas por punto de trabajo y por rango de fechas, de modo que el Jefe pueda comparar el avance entre distintos puntos de venta durante el evento.

6. Consideración de acceso

El Dashboard es una ruta protegida (`/dashboard`, `roles={['jefe']}`) tanto a nivel de frontend (`ProtectedRoute`) como de backend (`authorize('jefe')` en `/api/tickets/stats`), consistente con la matriz de permisos consolidada en la Semana 4.

7. Conclusiones del día

El Dashboard administrativo queda funcional, presentando de forma visual el estado general del canje (avance, evolución diaria, distribución por punto de trabajo), consumiendo directamente los reportes agregados desarrollados el día anterior.

Observaciones: Sin observaciones.

Optimización de Consultas y Rendimiento

Actividad N°: 29 **Fecha:** 09/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Revisar y consolidar las optimizaciones de rendimiento aplicadas al backend, verificando que el sistema soporte el uso concurrente de varios puntos de venta sin degradarse (RNF-06, RNF-07).

2. Problemas de rendimiento identificados originalmente

- Error de cast en `checkTicketsChangesForStaff` al intentar buscar un `PuntoVenta` por `ObjectId` cuando `puntoTrabajo` en realidad es un `String` (nombre del punto).
- Ausencia de índices en campos consultados con alta frecuencia (`Ticket`, `updatedAt`).
- Consultas sin límite de tiempo, con riesgo de dejar el servidor colgado ante una consulta lenta.
- Rate limiting demasiado estricto (100 solicitudes cada 15 minutos), que bloqueaba el polling de verificación de cambios.
- Pool de conexiones de MongoDB pequeño para el número de puntos de venta concurrentes esperado.

3. Optimizaciones implementadas

Optimización	Detalle
Corrección de búsqueda por nombre	<code>PuntoVenta.findOne({ nombre: userPuntoTrabajo, activo: true })</code> en vez de buscar por <code>_id</code>
Índices agregados al modelo <code>Ticket</code>	<code>Ticket</code> (localidad), <code>updatedAt</code> (desc), compuesto <code>Ticket + updatedAt</code> , además de los índices de búsqueda por nombre/email/cédula ya definidos
Timeouts explícitos	<code>.maxTimeMS(10000)</code> en <code>find()</code> , <code>.maxTimeMS(5000)</code> en <code>countDocuments()</code> , <code>.maxTimeMS(3000)</code> en verificaciones ligeras de cambios (<code>check-changes</code>)
Rate limiting ajustado	De 100 solicitudes/15 min a 200 solicitudes/minuto, excluyendo explícitamente <code>/check-changes</code> y <code>/health</code>
Configuración de conexión MongoDB	<code>maxPoolSize: 20</code> , <code>minPoolSize: 5</code> , <code>retryReads: true</code> , <code>retryWrites: true</code> , <code>connectTimeoutMS</code> , <code>heartbeatFrequencyMS</code> , IPv4 forzado (<code>family: 4</code>)
Consultas en paralelo	<code>Promise.all</code> para ejecutar la búsqueda de documentos y el conteo total de forma simultánea, en vez de secuencial
Uso de <code>.lean()</code>	En todas las consultas de solo lectura (listados de tickets, verificación de cambios), evitando hidratar documentos Mongoose completos innecesariamente
Manejo de errores de timeout	Respuesta <code>503</code> con <code>retry: true</code> cuando una consulta excede su tiempo máximo, en vez de un error genérico <code>500</code>

Optimización	Detalle
Caché en memoria (<code>node-cache</code>)	Middleware <code>cacheMiddleware</code> con dos niveles de TTL (5 min general, 1 min para datos que cambian rápido) e invalidación selectiva por patrón (<code>invalidateCache</code>)

4. Resultados medidos (según seguimiento del equipo)

Métrica	Antes	Después	Mejora
Tiempo de respuesta de queries	2-5 s	0.5-1 s	~75%
Solicitudes bloqueadas por rate limit	~20%	<1%	~95%
Cuelgues del servidor	Frecuentes	Casi nulos	~98%
Uso de memoria	Alto	Moderado	~35%
Consultas por segundo soportadas	~10	~30-40	~300%

5. Verificación de índices

Se cuenta con un script dedicado (`backend/src/scripts/createIndexes.js`) que crea (o verifica) los índices necesarios, incluyendo un índice de texto compuesto sobre nombre, apellido, email, localidad, Ticket ID, Transaction ID y cédula, útil como respaldo cuando la búsqueda por regex no es suficiente.

6. Conclusiones del día

El sistema queda optimizado para el escenario real de uso (varios puntos de venta consultando y canjeando de forma concurrente durante el evento), con mejoras medibles en tiempo de respuesta, estabilidad ante consultas lentas y capacidad de solicitudes por segundo.

Observaciones: Sin observaciones.